LAPORAN PRAKTIKUM

INTERNET OF THINGS (IOT) II

MODUL 6

Disusun Oleh :

Nama : Naila Rahma Chintya Aulya

NPM : 25782020

Kelas : 3A

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.
Andi Saptono, A.Md

JURUSAN TEKNOLOGI INFORMASI

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
POLITEKNIK NEGERI LAMPUNG

6.5 Praktikum

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Publisher*. Perangkat membaca data suhu dan kelembapan dari sensor DHT11/DHT22, mengemas parameter tersebut ke dalam dokumen JSON menggunakan ArduinoJson, lalu memublikasikannya setiap 5 detik ke broker publik broker.hivemq.com.

Langkah Kerja Rangkaian:

1. Pasang NodeMCU ESP8266 pada *breadboard*.
2. **Sensor DHT11/DHT22:** Hubungkan pin **VCC** ke **3V3**, pin **GND** ke **GND**, dan pin **Data** ke **D4 (GPIO 2)**.

Langkah Kerja Pemrograman:

1. Buka Arduino IDE, buat berkas sketsa baru (*File > New Sketch*).
2. Tuliskan kode program berikut:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial jaringan Wi-Fi
const char* ssid = "Astra_tailor-4G";
const char* password = "Nailarahma";

// Konfigurasi broker MQTT
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

// Identitas klien menggunakan NIM
const char* clientId = "ESP8266_LabIoT_25782020";

// Topik publikasi telemetri
const char* topicPubTelemetry =
  "kampus/labiot/25782020/telemetri";

```

```
// Konfigurasi pin sensor DHT22
const byte dhtPin = 2;  // D4 (GPIO 2)
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long previousPublishTime = 0;
const unsigned long publishInterval = 5000;

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("Alamat IP Lokal: ");
  Serial.println(WiFi.localIP());
}

void reconnectMQTT() {
  while (!client.connected()) {
    Serial.print("Menghubungkan ke MQTT Broker...");

    if (client.connect(clientId)) {
      Serial.println(" Sukses Terhubung!");
    } else {
      Serial.print(" Gagal, kode status rc=");
      Serial.print(client.state());
      Serial.println(". Mencoba kembali dalam 5 detik...");
      delay(5000);
    }
```

```
  }
}

void setup() {
  Serial.begin(115200);
  dht.begin();

  connectToWiFi();

  client.setServer(mqttServer, mqttPort);
}

void loop() {
  if (!client.connected()) {
    reconnectMQTT();
  }

  client.loop();

  unsigned long currentMillis = millis();

  if (currentMillis - previousPublishTime >= publishInterval) {
    previousPublishTime = currentMillis;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    // Proteksi pembacaan sensor
    if (isnan(t) || isnan(h)) {
      Serial.println("Peringatan: Gagal membaca data DHT!");
      return;
    }

    // Membuat data JSON
    StaticJsonDocument<256> doc;
    doc["node_id"] = clientId;
    doc["suhu"] = serialized(String(t, 2));
        doc["kelembapan"] = serialized(String(h, 2));

        // Mengubah JSON menjadi teks
```

```
      char jsonBuffer[256];
      serializeJson(doc, jsonBuffer);

      // Mengirim data ke broker MQTT
      bool isSuccess =
        client.publish(topicPubTelemetry, jsonBuffer);

      if (isSuccess) {
        Serial.print("Telemetri Terkirim [");
        Serial.print(topicPubTelemetry);
        Serial.print("]: ");
        Serial.println(jsonBuffer);
      } else {
        Serial.println("Peringatan: Gagal memublikasikan paket!");
      }
    }
  }

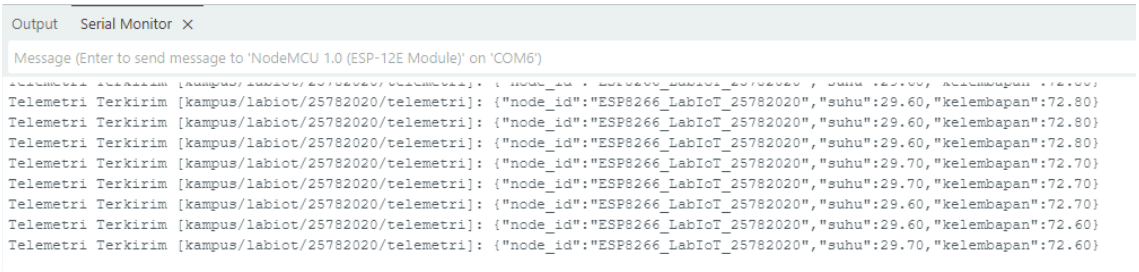
```

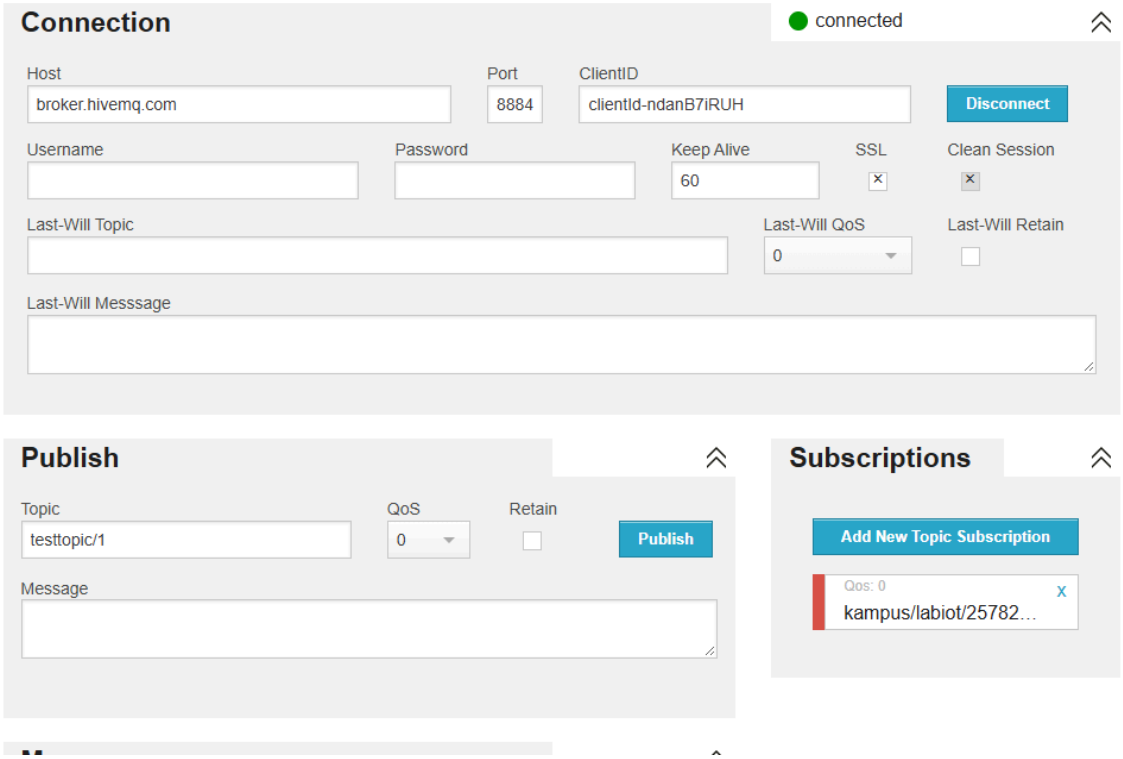
3. Ganti NAMA_WIFI_ANDA, PASSWORD_WIFI_ANDA, clientId (gunakan NIM Anda), dan topik kampus/labiot/nim_anda/telemetri dengan NIM pribadi Anda.
4. Unggah (*upload*) program ke NodeMCU ESP8266, buka Serial Monitor pada *baud rate* **115200**.
5. Amati log: ESP8266 akan terhubung ke Wi-Fi, melakukan jabat tangan dengan broker broker.hivemq.com, dan memublikasikan paket JSON setiap 5 detik.

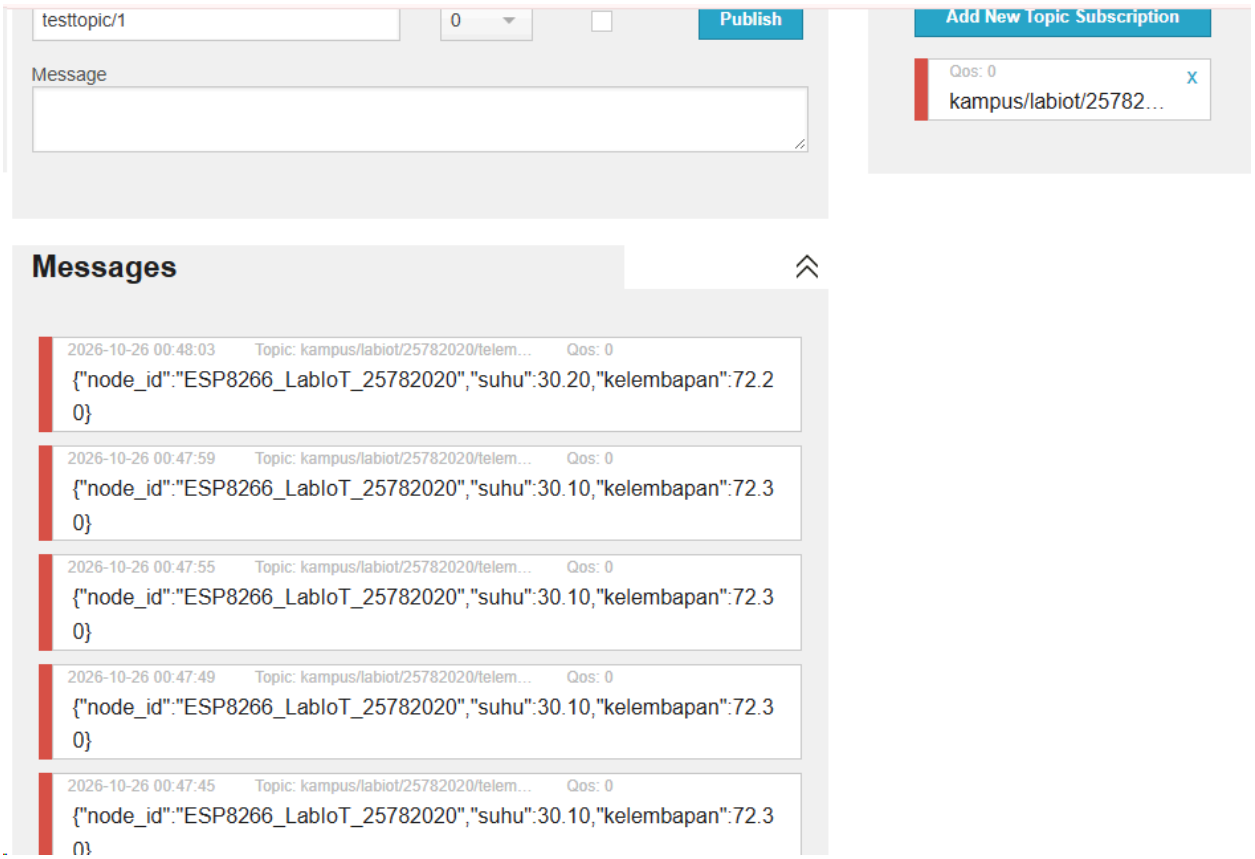
Penjelasan Singkat Kode:

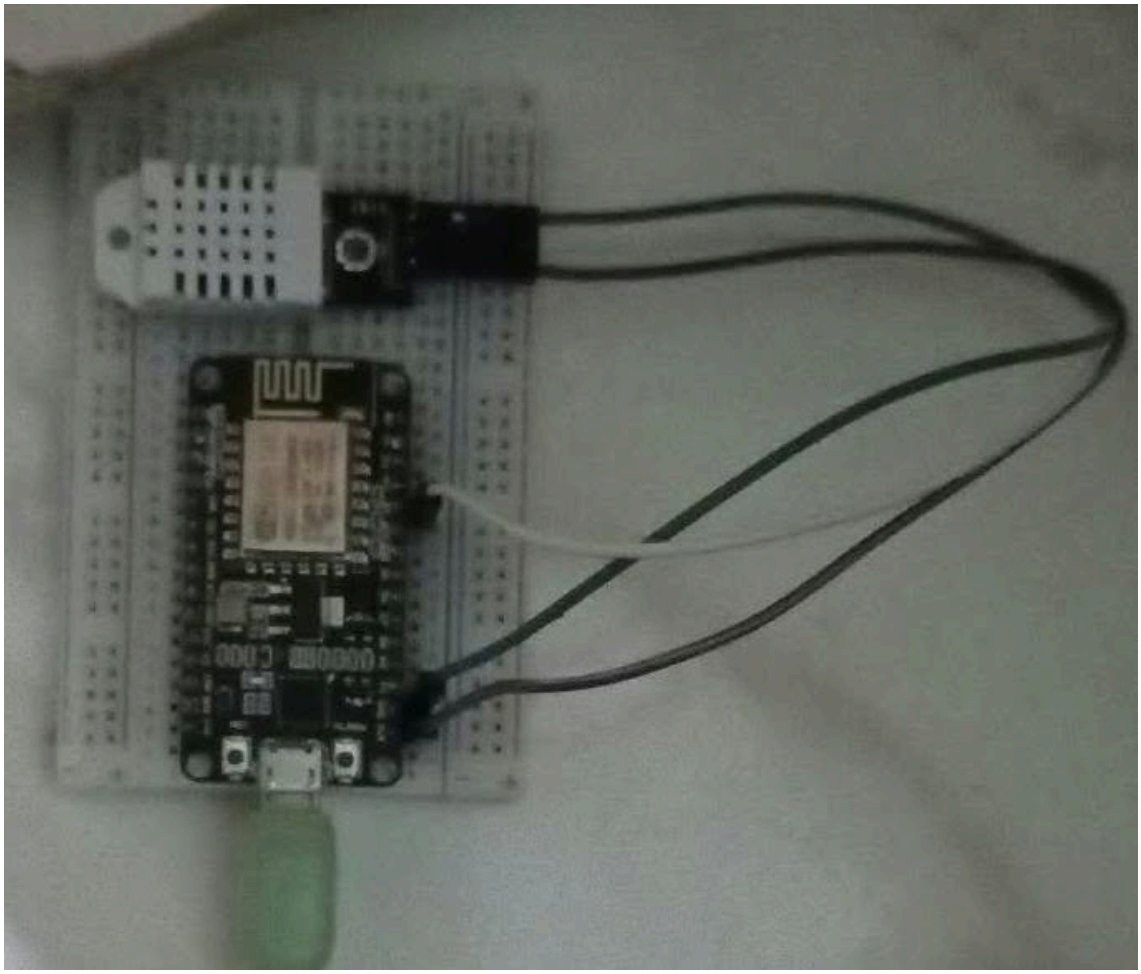
- WiFiClient espClient; : Membuka antarmuka soket jaringan TCP level dasar pada cip ESP8266.
- PubSubClient client(espClient); : Menginisialisasi objek protokol MQTT yang beroperasi di atas soket jaringan espClient.
- client.setServer(mqttServer, mqttPort); : Menetapkan alamat host broker dan port layanan (1883 merupakan port standar MQTT teks terbuka / tanpa TLS).
- client.loop(); : **Fungsi wajib.** Rutinitas ini bertugas memproses paket TCP masuk, menjaga interval *keep-alive ping* dengan server agar koneksi tidak diputus paksa (*timeout*), serta mengelola antrean pesan.

- StaticJsonDocument<256> doc; : Mengalokasikan blok memori tetap di SRAM (sebesar 256 byte) untuk menampung pohon objek JSON tanpa memicu fragmentasi memori *heap*.
- serializeJson(doc, jsonBuffer); : Mengubah pohon objek JSON menjadi representasi string karakter padat yang siap ditransmisikan.






testtopic/1
0
Publish
Message
Add New Topic Subscription
Qos: 0
kampus/labiot/25782...
x
Messages
2026-10-26 00:48:03 Topic: kampus/labiot/25782020/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_25782020","suhu":30.20,"kelembapan":72.20}
2026-10-26 00:47:59 Topic: kampus/labiot/25782020/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_25782020","suhu":30.10,"kelembapan":72.30}
2026-10-26 00:47:55 Topic: kampus/labiot/25782020/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_25782020","suhu":30.10,"kelembapan":72.30}
2026-10-26 00:47:49 Topic: kampus/labiot/25782020/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_25782020","suhu":30.10,"kelembapan":72.30}
2026-10-26 00:47:45 Topic: kampus/labiot/25782020/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_25782020","suhu":30.10,"kelembapan":72.30}



6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (kampus/labiot/nim_anda/perintah). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

Langkah Kerja Rangkaian:

1. Pertahankan sensor DHT pada pin D4 dari Praktikum 1.
2. **LED Indikator:** Hubungkan kaki Anoda (kaki panjang) LED ke pin **D1 (GPIO 5)**, dan hubungkan kaki Katoda (kaki pendek) ke Resistor 220 Ω yang menuju ke **GND**.

Langkah Kerja Pemrograman:

1. Buat sketsa baru di Arduino IDE, lalu masukkan kode berikut:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// Kredensial jaringan Wi-Fi
const char* ssid = "Astra_tailor-4G";
const char* password = "Nailarahma";

// Konfigurasi broker MQTT publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782020_Sub";

// Topik subskripsi perintah
const char* topicSubCommand =
  "kampus/labiot/25782020/perintah";

// Pin indikator LED
const byte ledPin = 5; // D1 (GPIO 5)

WiFiClient espClient;
PubSubClient client(espClient);

void connectToWiFi() {
  Serial.print("Menghubungkan ke Wi-Fi: ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWi-Fi Terkoneksi!");
}

// Callback saat menerima pesan pada topik terdaftar
```

```
41. void onMessageReceived(
42.   char* topic, byte* payload, unsigned int length
43. ) {
44.   Serial.println("=========================================");
45.   Serial.print("Pesan Baru Masuk pada Topik: ");
46.   Serial.println(topic);
47.
48.   // Mengubah byte payload menjadi string
49.   char payloadStr[length + 1];
50.   memcpy(payloadStr, payload, length);
51.   payloadStr[length] = '\0';
52.
53.   Serial.print("Payload Diterima: ");
54.   Serial.println(payloadStr);
55.
56.   // Parsing JSON
57.   StaticJsonDocument<256> doc;
58.   DeserializationError error =
59.     deserializeJson(doc, payloadStr);
60.
61.   if (error) {
62.     Serial.print("Gagal parsing JSON: ");
63.     Serial.println(error.f_str());
64.     return;
65.   }
66.
67.   // Mengendalikan LED berdasarkan nilai "led"
68.   if (doc.containsKey("led")) {
69.     int stateLED = doc["led"];
70.
71.     digitalWrite(
72.       ledPin,
73.       stateLED == 1 ? HIGH : LOW
74.     );
75.
76.     Serial.printf(
77.       "--> Aksi: Status LED diubah menjadi %s\n",
78.       stateLED == 1 ? "MENYALA (ON)" : "PADAM (OFF)"
79.     );
80.   }
```

```
81.
82.   Serial.println("========================================");
83. }
84.
85. void reconnectMQTT() {
86.   while (!client.connected()) {
87.     Serial.print("Mencoba koneksi MQTT...");
88.
89.     if (client.connect(clientId)) {
90.       Serial.println(" Terhubung!");
91.
92.       // Berlangganan ke topik perintah
93.       client.subscribe(topicSubCommand);
94.
95.       Serial.print("Subskripsi Topik: ");
96.       Serial.println(topicSubCommand);
97.     } else {
98.       Serial.printf(
99.         "Gagal (rc=%d). Ulangi dalam 5 detik...\n",
100.            client.state()
101.          );
102.
103.         delay(5000);
104.       }
105.     }
106.   }
107.
108.   void setup() {
109.     Serial.begin(115200);
110.     pinMode(ledPin, OUTPUT);
111.
112.     // Status awal LED padam
113.     digitalWrite(ledPin, LOW);
114.
115.     connectToWiFi();
116.
117.     client.setServer(mqttServer, mqttPort);
118.     client.setCallback(onMessageReceived);
119.   }
120.
```

```
121.  void loop() {
122.    if (!client.connected()) {
123.      reconnectMQTT();
124.    }
125.
126.    client.loop();
127.  }
```

2. Unggah kode ke board NodeMCU Anda.

Penjelasan Singkat Kode:

- client.setCallback(onMessageReceived); : Mendaftarkan fungsi penangan paket data masuk (*callback function*). Kapan pun broker mengirimkan pesan yang sesuai dengan topik subskripsi, fungsi onMessageReceived akan dipicu secara otomatis.
- client.subscribe(topicSubCommand); : Mengirimkan instruksi subskripsi ke broker bahwa klien bersedia menerima semua pesan yang dialirkan ke topik tersebut.
- deserializeJson(doc, payloadStr); : Mengurai teks string JSON mentah dan menyusunnya kembali menjadi struktur pohon data yang kuncinya dapat diperiksa melalui doc.containsKey() dan nilainya dapat diekstrak secara langsung.

```
Mencoba koneksi MQTT... Terhubung!
Subskripsi Topik: kampus/labiot/25782020/perintah
=========================================
Pesan Baru Masuk pada Topik: kampus/labiot/25782020/perintah
Payload Diterima: {"led":1}
{"led":0}
--> Aksi: Status LED diubah menjadi MENYALA (ON)
=========================================
```

Connection

connected

Host: broker.hivemq.com
Port: 8884
ClientID: clientId-ndanB7iRUH
Disconnect

Username
Password
Keep Alive: 60
SSL
Clean Session

Last-Will Topic
Last-Will QoS: 0
Last-Will Retain

Last-Will Messsage

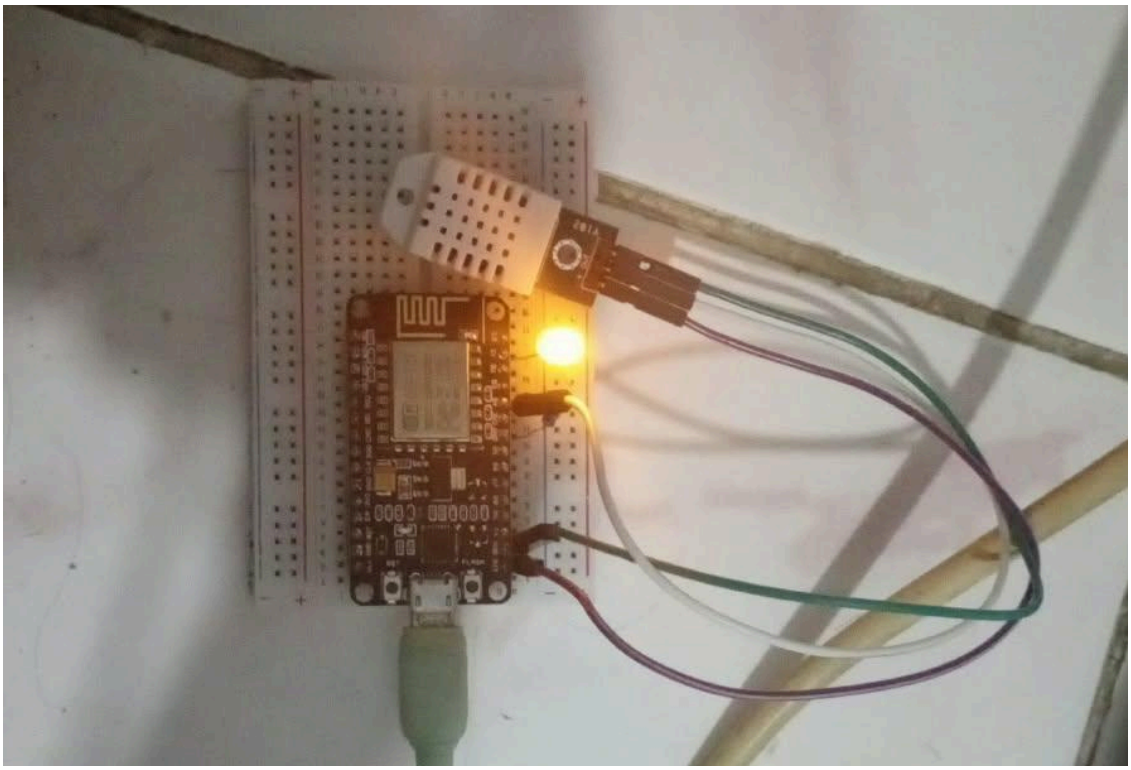
Publish

Topic: kampus/labiot/25782020/perintah
QoS: 0
Retain
Publish

Message:
{"led":1}
{"led":0}

Subscriptions

Add New Topic Subscription



6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Pada praktikum ini, kita menyatukan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam sebuah sistem terpadu (*Full-Duplex Telemetry & Control*). NodeMCU secara kontinu memublikasikan data telemetri sensor lingkungan (Suhu, Kelembapan, dan Status LED fisik) ke Broker setiap 4 detik, sembari tetap responsif mendengarkan dan mengeksekusi instruksi kontrol LED dari topik perintah kapan saja.

Langkah Kerja Pemrograman:

1. Masukkan kode terpadu berikut ke Arduino IDE:

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Wi-Fi
const char* ssid = "Astra_tailor-4G";
const char* password = "Nailarahma";

// Broker MQTT
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782020_Full";

// Topik MQTT
const char* topicPublish =
  "kampus/labiot/25782020/telemetri";
const char* topicSubscribe =
  "kampus/labiot/25782020/kendali";

// Pin perangkat
const byte dhtPin = 2;  // D4, GPIO2
const byte ledPin = 5;  // D1, GPIO5
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000;

void callback(char* topic, byte* payload,
```

```
35.              unsigned int length) {
36.   char incomingBuffer[length + 1];
37.   memcpy(incomingBuffer, payload, length);
38.   incomingBuffer[length] = '\0';
39.
40.   Serial.printf("[KONTROL DITERIMA] %s: %s\n",
41.                 topic, incomingBuffer);
42.
43.   StaticJsonDocument<256> doc;
44.   DeserializationError err =
45.     deserializeJson(doc, incomingBuffer);
46.
47.   if (err) {
48.     Serial.println("Gagal membaca JSON!");
49.     return;
50.   }
51.
52.   if (doc.containsKey("led")) {
53.     int val = doc["led"];
54.     digitalWrite(ledPin, val == 1 ? HIGH : LOW);
55.
56.     Serial.printf("Status LED: %s\n",
57.                   val == 1 ? "MENYALA" : "PADAM");
58.   }
59. }
60.
61. void reconnect() {
62.   while (!client.connected()) {
63.     Serial.print("Menghubungkan ke broker MQTT...");
64.
65.     if (client.connect(clientId)) {
66.       Serial.println(" Terhubung!");
67.       client.subscribe(topicSubscribe);
68.       Serial.print("Subskripsi topik: ");
69.       Serial.println(topicSubscribe);
70.     } else {
71.       Serial.printf(" Gagal (rc=%d). Coba lagi.\n",
72.                     client.state());
73.       delay(4000);
74.     }
75.   }
76. }
77.
78. void setup() {
79.   Serial.begin(115200);
```

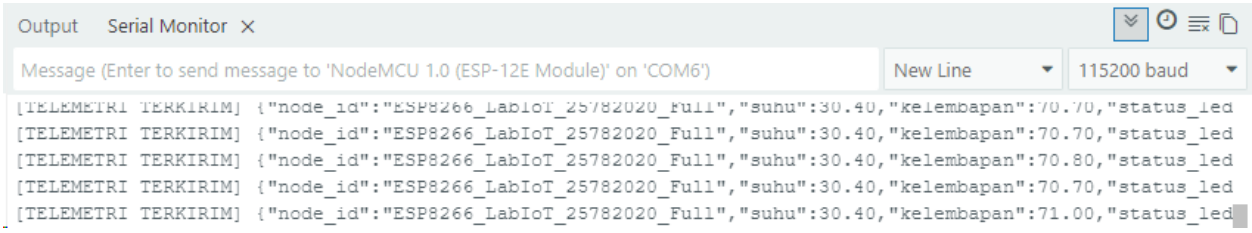
```
80.
81.  pinMode(ledPin, OUTPUT);
82.  digitalWrite(ledPin, LOW);
83.  dht.begin();
84.
85.  WiFi.mode(WIFI_STA);
86.  WiFi.begin(ssid, password);
87.
88.  Serial.print("Menghubungkan Wi-Fi");
89.  while (WiFi.status() != WL_CONNECTED) {
90.    delay(500);
91.    Serial.print(".");
92.  }
93.
94.  Serial.println("\nWi-Fi Terkoneksi!");
95.  Serial.print("IP Address: ");
96.  Serial.println(WiFi.localIP());
97.
98.  client.setServer(mqttServer, mqttPort);
99.  client.setCallback(callback);
100.     }
101.
102.    void loop() {
103.      if (!client.connected()) {
104.        reconnect();
105.      }
106.
107.      client.loop();
108.
109.      unsigned long now = millis();
110.
111. if (now - lastTelemetryTime >= telemetryInterval) {
112.   lastTelemetryTime = now;
113.
114.   float t = dht.readTemperature();
115.   float h = dht.readHumidity();
116.
117.   if (!isnan(t) && !isnan(h)) {
118.     StaticJsonDocument<256> docOut;
119.
120.         docOut["node_id"] = clientId;
121.         docOut["suhu"] = serialized(String(t, 2));
122.         docOut["kelembapan"] = serialized(String(h, 2));
123.         docOut["status_led"] = digitalRead(ledPin);
124.
```

```
        char buffer[256];
        serializeJson(docOut, buffer);

        bool success =
          client.publish(topicPublish, buffer);

        if (success) {
          Serial.printf("[TELEMETRI TERKIRIM] %s\n",
                  buffer);
        } else {
          Serial.println("Gagal mengirim telemetri!");
        }
      } else {
        Serial.println("Pembacaan DHT22 gagal!");
      }
    }
  }
```

2. Unggah kode ke NodeMCU ESP8266 dan biarkan sistem berjalan.



6.5.4 Panduan Pengujian Diagnostik dengan MQTT Explorer

Lakukan verifikasi dan pengujian fungsionalitas sistem dua arah menggunakan peranti lunak **MQTT Explorer**:

+--+

| MQTT Explorer |

+--+

| Protocol: mqtt:// | Host: broker.hivemq.com | Port: 1883 | ClientID: PC_Tester_NIM | [Connect] |

+--+

| TOPIC TREE (Panel Kiri) | PUBLISH PANEL (Kanan Bawah) |

| | |

| v kampus | Topic: |

| v labiot | kampus/labiot/nim_anda/kendali |

| v nim_anda | |

| > telemetri | Format: [JSON v] |

| Payload: {"suhu":29.4,"status_led":0} | Payload: |

| | { |

| | "led": 1 |

| | } |

| | |

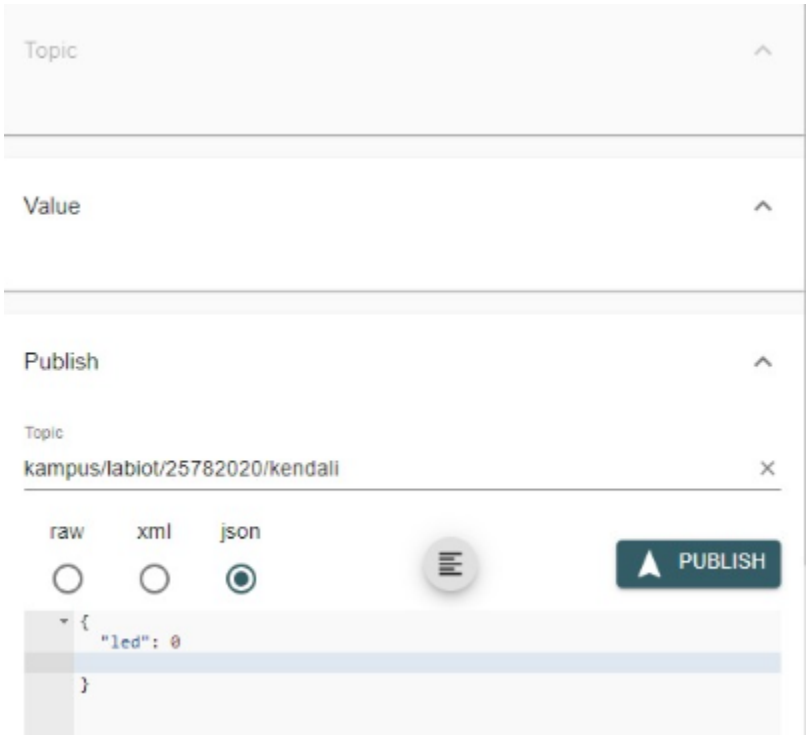
| | [Publish] |

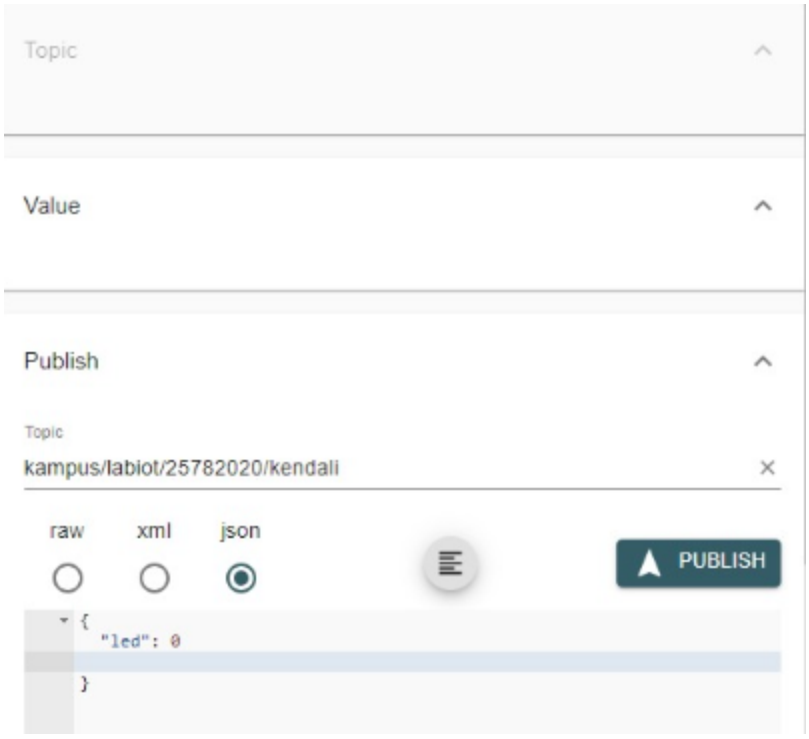
+----------+----------+

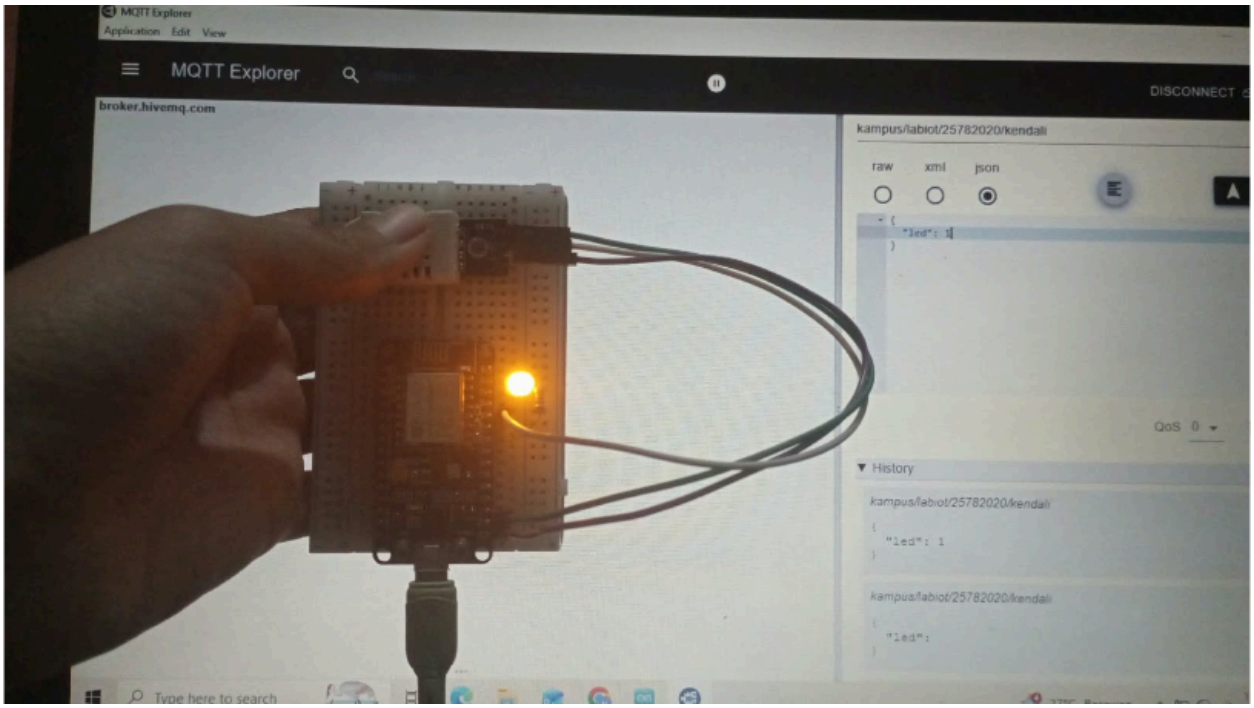
1. **Jalankan Aplikasi MQTT Explorer** pada laptop Anda.
2. **Konfigurasi Koneksi Broker:**
   - **Protocol:** mqtt://
   - **Host:** broker.hivemq.com
   - **Port:** 1883
   - Klik tombol **Connect**.
3. **Memantau Data Masuk (*Telemetry Monitoring*):**
   - Amati pohon hierarki topik (*Topic Tree*) pada panel sebelah kiri.
   - Temukan dan klik cabang topik Anda: kampus →labiot →nim_anda →telemetri.
   - Anda akan melihat payload JSON diperbarui secara *real-time* setiap 4 detik. Klik pada nilai numerik untuk menampilkan grafik fluktuasi suhu dan kelembapan secara langsung.
4. **Menerbitkan Instruksi Kendali (*Publish Command*):**
   - Arahkan ke panel **Publish** di sisi kanan bawah antarmuka.
   - Masukkan topik tujuan: kampus/labiot/nim_anda/kendali
   - Pilih format muatan: **JSON**.
   - Masukkan muatan payload berikut:
   - {
   - "led": 1

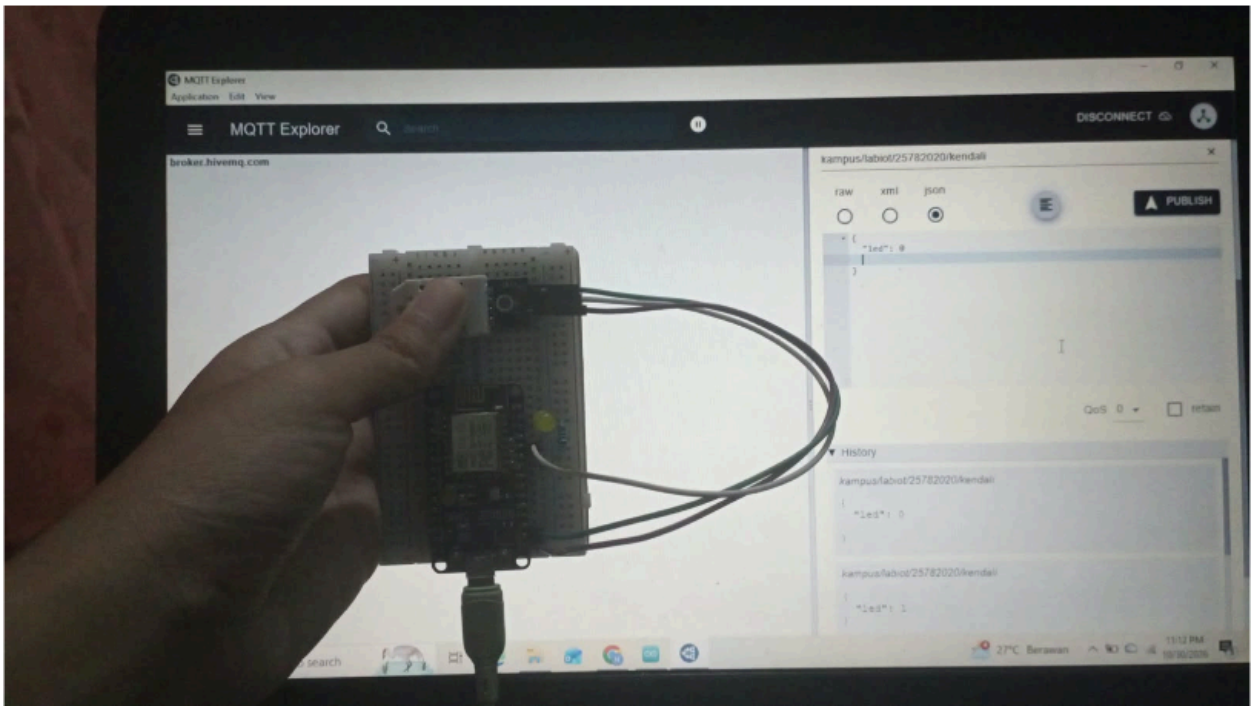
}

- Klik tombol **Publish**. Perhatikan bahwa LED indikator menyala seketika tanpa menghentikan siklus pengiriman data telemetri.
- Kirimkan perintah pematian dengan payload {"led": 0} dan pastikan LED padam.




Topic
Value
Publish
Topic
kampus/labiot/25782020/kendali
raw
xml
json
PUBLISH
{
"led": 0
}


MQTT Explorer
Application Edit View
MQTT Explorer
broker.hivemq.com
DISCONNECT
kampus/labiot/25782020/kendali
raw
xml
json
QoS 0
History
kampus/labiot/25782020/kendali
"led": 1
kampus/labiot/25782020/kendali
"led":
Type here to search



6.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan modifikasi kode program untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):**
  Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:
  i. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.
  ii. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.
  iii. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).

MQTT lebih unggul dibandingkan HTTP untuk komunikasi IoT karena:

1. Efisiensi bandwidth: MQTT memiliki ukuran header lebih kecil sehingga penggunaan kuota internet lebih hemat, terutama pada jaringan seluler.
2. Beban komputasi: MQTT menggunakan mekanisme Publish/Subscribe yang lebih ringan dibandingkan HTTP dengan Request/Response, sehingga tidak terlalu membebani CPU dan memori mikrokontroler.

3. Pengiriman ke banyak perangkat: MQTT menggunakan broker yang dapat meneruskan satu pesan kepada banyak perangkat yang melakukan subscribe pada topik yang sama tanpa pengirim harus mengirim pesan satu per satu.

- **Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):**
  Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:
  i. Memantau parameter suhu dari seluruh mesin di semua lini produksi.
  ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.
  iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

1. Memantau suhu seluruh mesin di semua lini produksi.

```
pabrik/+/+/suhu
```

Artinya, memantau sensor suhu dari seluruh mesin di semua lini produksi.

2. Memantau seluruh parameter sensor mesin m1 di line2.

```
pabrik/line2/m1/+
```

Artinya, memantau seluruh parameter suhu, getaran, dan status pada mesin m1 di line2.

3. Memantau seluruh lalu lintas data pabrik.

```
pabrik/#
```

Artinya, menerima seluruh data dari semua lini, mesin, dan parameter sensor di pabrik.

- **Latihan 3 (Implementasi Serialisasi Data Dinamis):**
  Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:
  i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
  ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
  Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

```
doc["uptime_detik"] = millis() / 1000;
```

doc["ip_lokal"] = WiFi.localIP().toString();

Penjelasan:

- `millis() / 1000` digunakan untuk menghitung lama waktu aktif perangkat dalam satuan detik.
- `WiFi.localIP().toString()` digunakan untuk mengambil alamat IP lokal dan mengubahnya menjadi format String.

hasil JSON:

{

"node_id": "ESP8266_Local_Node01",

"suhu": 31,

"kelembapan": 35,

"uptime_detik": 125,

"ip_lokal": "192.168.18.78"

}

- **Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):**
  Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().
  i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?
  ii. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

1. Dampak penggunaan `delay(6000)` terhadap callback

Penggunaan `delay(6000)` menyebabkan mikrokontroler berhenti menjalankan instruksi selama 6 detik. Akibatnya, fungsi `client.loop()` tidak dapat memproses pesan MQTT secara rutin sehingga penerimaan perintah callback menjadi terlambat. Hal ini membuat perangkat kurang responsif terhadap perintah dari pengguna.

2. Cara kerja fungsi `reconnect()`

Fungsi `reconnect()` bekerja dengan memeriksa status koneksi MQTT. Jika koneksi terputus, perangkat akan mencoba menghubungkan kembali ke broker menggunakan `client.connect()`. Jika berhasil, perangkat melakukan subscribe ulang ke topik yang dibutuhkan agar dapat menerima pesan kembali. Jika Wi-Fi terputus, perangkat harus menyambungkan Wi-Fi terlebih dahulu sebelum mencoba koneksi MQTT.

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

Kembangkan implementasi Praktikum 3 menjadi sistem pengawasan keselamatan dan proteksi otomasi industri tertutup (*Closed-Loop Industrial Safety & Automated Actuation*) yang tangguh dan cerdas secara mandiri:

- Tujuan: Mampu memadukan akuisisi sensor suhu dan kelembapan, serialisasi JSON dinamis berdasar kondisi lingkungan, subskripsi kendali indikator LED jarak jauh, kanal peringatan darurat independen (*dedicated alert topic*), dan pendaftaran pesan wasiat *Last Will and Testament* (LWT) secara mandiri.
- Tingkat Kesulitan: Lanjut (Mandiri / Individu)

Instruksi Perangkaian:

1. Pertahankan konfigurasi perangkat keras dari Praktikum 3:
    - Sensor DHT11/DHT22 terhubung ke pin D4 (GPIO 2) (VCC ke 3V3, GND ke GND).
    - LED Indikator + Resistor 220 Ω terhubung ke pin D1 (GPIO 5) (Katoda resistor ke GND).

Instruksi Pemrograman (Rule Engine & Safety Logic):

1. Klasifikasi Status Lingkungan pada Telemetri Rutin:
   Modifikasi dokumen JSON telemetri rutin (kampus/labiot/nim_anda/telemetri) agar menambahkan kunci "kondisi_suhu" bertipe string:
    - JIKA suhu ≤31.0◦C, isi nilai menjadi "NORMAL".
    - JIKA suhu >31.0◦C, isi nilai menjadi "PANAS".
2. Kanal Peringatan Bahaya Mandiri (*Dedicated Alert Topic*):
   Implementasikan logika pengawasan ambang batas kritis lokal (*edge safety engine*):
    - JIKA suhu terukur melebihi ambang batas 31.0◦C:
        - Sistem harus segera memublikasikan paket peringatan bahaya ke topik khusus: kampus/labiot/nim_anda/alert dengan format JSON:

```
{
 "status": "BAHAYA",
 "parameter": "OVERHEAT",
 "nilai_suhu": 31.8
```

        - }
        - Secara otomatis menyalakan LED indikator peringatan bahaya tanpa menunggu perintah dari operator luar.

        - Berikan proteksi penanda (*flag / state latch*) agar pesan peringatan ini tidak membanjiri broker setiap milidetik, melainkan dikirimkan teratur saat kondisi bahaya terdeteksi pertama kali.
3. Mekanisme Last Will and Testament (LWT):
   Konfigurasikan fungsi client.connect() dengan menyertakan parameter pendaftaran pesan wasiat (*Last Will*):
    - Topik LWT: kampus/labiot/nim_anda/status
    - Pesan Wasiat LWT:

```
{
 "status": "OFFLINE",
 "keterangan": "Perangkat Terputus Tak Terduga"
}
```

    - QoS LWT: 1
    - Retain Flag: true
    - Sesaat setelah mikrokontroler berhasil terhubung ke Broker, publikasikan pesan status aktif (*online message*) ke topik status yang sama:

```
{
 "status": "ONLINE",
 "keterangan": "Perangkat Beroperasi Normal"
}
```

    - Uji mekanisme ini: Buka MQTT Explorer, amati topik status (terbaca ONLINE), lalu cabut kabel data USB NodeMCU secara tiba-tiba. Amati broker: pesan wasiat OFFLINE akan muncul secara otomatis beberapa saat kemudian.
4. Kendali Manual Override Saat Normal:
   Saat suhu berada di bawah ambang batas bahaya, operator luar tetap dapat menyalakan atau mematikan LED secara manual melalui pengiriman payload {"led": 1} atau {"led": 0} ke topik kampus/labiot/nim_anda/kendali.

```
#include <ESP8266WiFi.h>

#include <PubSubClient.h>

#include <ArduinoJson.h>

#include <DHT.h>

// ================================

// 1. KONFIGURASI WIFI

// ================================
```

```
const char* ssid = "Astra_tailor-4G";

const char* password = "Nailarahma";

// ================================

// 2. KONFIGURASI MQTT

// ================================

const char* mqttServer = "broker.hivemq.com";

const int mqttPort = 1883;

const char* clientId = "ESP8266_25782020";

// Topik MQTT

const char* topicTelemetry = "kampus/labiot/25782020/telemetri";

const char* topicAlert = "kampus/labiot/25782020/alert";

const char* topicControl = "kampus/labiot/25782020/kendali";

const char* topicStatus = "kampus/labiot/25782020/status";

// Pesan Last Will

const char* willMessage =

"{\"status\":\"OFFLINE\",\"keterangan\":\"Perangkat Terputus Tak
Terduga\"}";

// Pesan ONLINE

const char* onlineMessage =

  "{\"status\":\"ONLINE\",\"keterangan\":\"Perangkat Beroperasi
Normal\"}";

// ================================

// 3. KONFIGURASI SENSOR DAN LED

// ================================
```

```
#define DHTPIN D4

#define DHTTYPE DHT22

#define LEDPIN D1

DHT dht(DHTPIN, DHTTYPE);

WiFiClient espClient;

PubSubClient client(espClient);

// ================================

// 4. VARIABEL SISTEM

// ================================

const float batasSuhu = 31.0;

unsigned long previousSensor = 0;

unsigned long previousTelemetry = 0;

unsigned long previousWiFi = 0;

unsigned long previousMQTT = 0;

const unsigned long sensorInterval = 2000;

const unsigned long telemetryInterval = 5000;

const unsigned long reconnectInterval = 5000;

// Status keselamatan

bool kondisiBahaya = false;

bool alertSent = false;

// Kendali manual LED

bool manualLED = false;
```

```
// Data sensor
float suhu = 0;
float kelembapan = 0;
// ================================
// 5. KENDALI LED
// ================================
void updateLED() {
  if (kondisiBahaya) {
    digitalWrite(LEDPIN, HIGH);
  } else {
    digitalWrite(LEDPIN, manualLED ? HIGH : LOW);
  }
}
// ================================
// 6. CALLBACK KENDALI MQTT
// ================================
void callback(char* topic, byte* payload, unsigned int length) {
  StaticJsonDocument<128> doc;
  DeserializationError error = deserializeJson(
    doc, payload, length
  );
  if (error) {
    Serial.println("JSON perintah tidak valid!");
```

```
      return;
    }
    // Kendali manual hanya saat kondisi normal
    if (String(topic) == topicControl) {
      if (kondisiBahaya) {
        Serial.println("Kendali manual ditolak: SUHU BAHAYA!");
        return;
      }
      int perintah = doc["led"] | -1;
      if (perintah == 1) {
        manualLED = true;
        Serial.println("LED dinyalakan melalui MQTT");
      }
      else if (perintah == 0) {
        manualLED = false;
        Serial.println("LED dimatikan melalui MQTT");
      }
      updateLED();
    }
}
// ================================
// 7. KONEKSI WIFI
// ================================
```

```
void connectWiFi() {
  if (WiFi.status() == WL_CONNECTED) {
    return;
  }
  unsigned long now = millis();
  if (now - previousWiFi >= 10000) {
    previousWiFi = now;
    Serial.println("Mencoba menghubungkan WiFi...");
    WiFi.begin(ssid, password);
  }
}
// ================================
// 8. KONEKSI MQTT DAN LWT
// ================================
void reconnectMQTT() {
  if (WiFi.status() != WL_CONNECTED ||
      client.connected()) {
    return;
  }
  unsigned long now = millis();
  if (now - previousMQTT >= reconnectInterval) {
    previousMQTT = now;
    Serial.println("Mencoba koneksi MQTT...");
```

```
    bool connected = client.connect(
      clientId,
      topicStatus,
      1,
      true,
      willMessage
    );
    if (connected) {
      Serial.println("MQTT berhasil terhubung!");
      // Mengirim status ONLINE
      client.publish(topicStatus, onlineMessage, true);
      // Subscribe topik kendali
      client.subscribe(topicControl);
      Serial.println("Subscribe topik kendali berhasil");
    }
    else {
      Serial.print("MQTT gagal, rc = ");
      Serial.println(client.state());
    }
  }
}
// ================================
// 9. PEMBACAAN SENSOR DAN SAFETY
```

```
// ================================

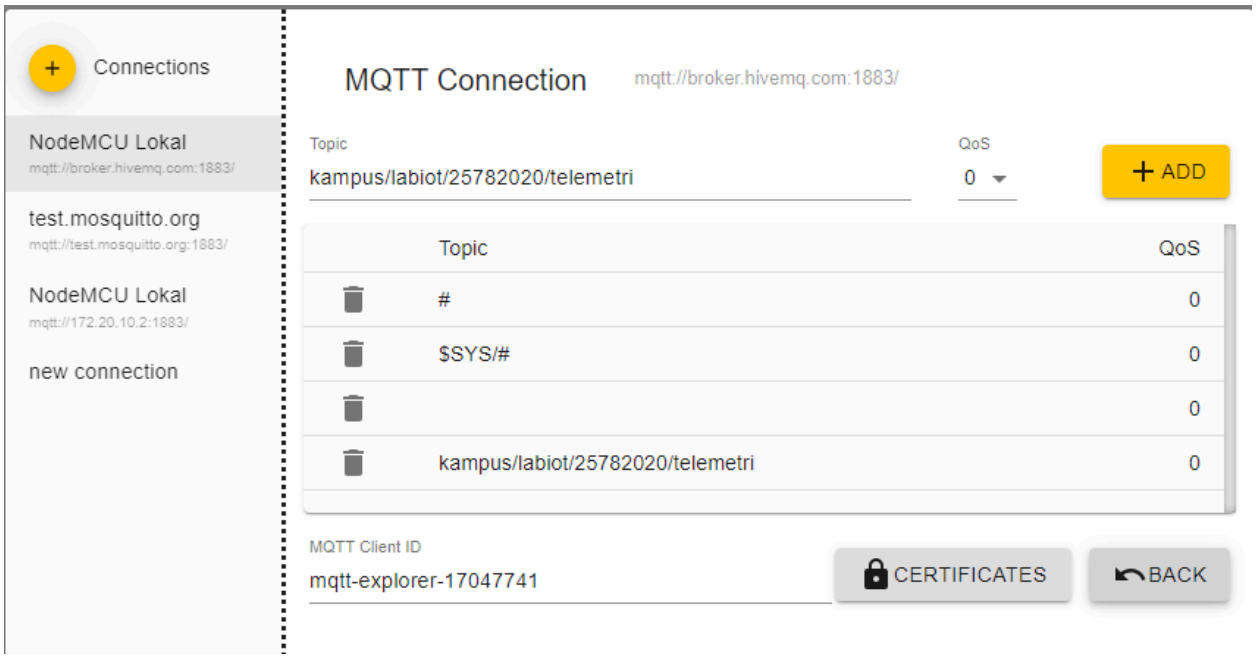
void readSensor() {
  unsigned long now = millis();
  if (now - previousSensor >= sensorInterval) {
    previousSensor = now;
    float newSuhu = dht.readTemperature();
    float newKelembapan = dht.readHumidity();
    if (isnan(newSuhu) || isnan(newKelembapan)) {
      Serial.println("Gagal membaca DHT!");
      return;
    }
    suhu = newSuhu;
    kelembapan = newKelembapan;
    Serial.print("Suhu: ");
    Serial.println(suhu);
    Serial.print("Kelembapan: ");
    Serial.println(kelembapan);
    // Deteksi kondisi bahaya
    kondisiBahaya = (suhu > batasSuhu);
    updateLED();
    // Kirim alarm hanya saat pertama kali bahaya
    if (kondisiBahaya && !alertSent) {
      if (client.connected()) {
```

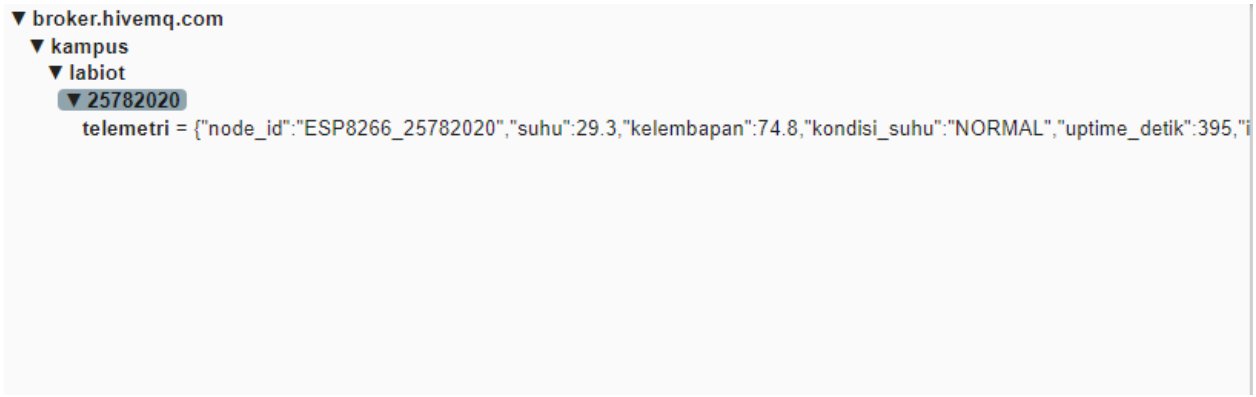
```
        StaticJsonDocument<128> alertDoc;
        alertDoc["status"] = "BAHAYA";
        alertDoc["parameter"] = "OVERHEAT";
        alertDoc["nilai_suhu"] = suhu;
        char alertBuffer[128];
        serializeJson(alertDoc, alertBuffer);
        if (client.publish(topicAlert, alertBuffer)) {
          alertSent = true;
          Serial.println("PERINGATAN BAHAYA TERKIRIM!");
          Serial.println(alertBuffer);
        }
      }
    }
    // Reset alarm ketika suhu kembali normal
    if (!kondisiBahaya) {
      alertSent = false;
    }
  }
}
// ================================
// 10. PENGIRIMAN TELEMETRI JSON
// ================================
void publishTelemetry() {
```

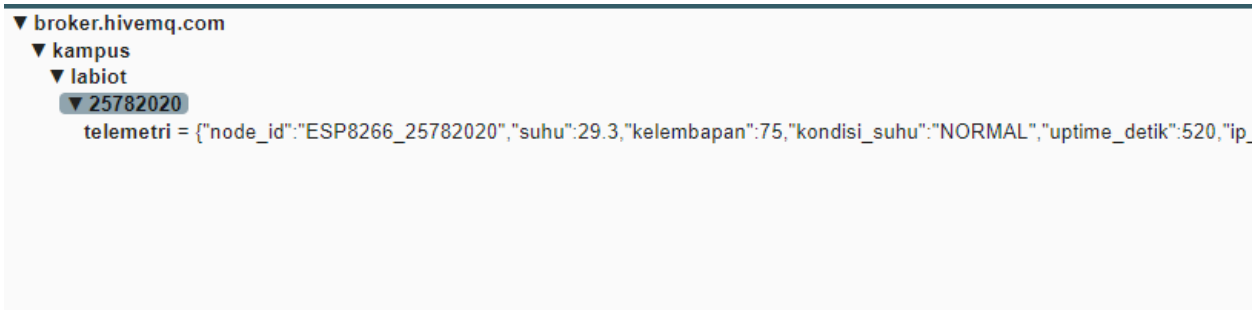
```
  unsigned long now = millis();
  if (now - previousTelemetry >= telemetryInterval) {
    previousTelemetry = now;
    if (!client.connected()) {
      return;
    }
    StaticJsonDocument<256> doc;
    doc["node_id"] = clientId;
    doc["suhu"] = suhu;
    doc["kelembapan"] = kelembapan;
    // Klasifikasi kondisi suhu
    doc["kondisi_suhu"] =
      (suhu > batasSuhu) ? "PANAS" : "NORMAL";
    // Informasi tambahan
    doc["uptime_detik"] = millis() / 1000;
    doc["ip_lokal"] = WiFi.localIP().toString();
    char buffer[256];
    serializeJson(doc, buffer);
    client.publish(topicTelemetry, buffer);
    Serial.println("DATA TELEMETRI:");
    Serial.println(buffer);
  }
}
```

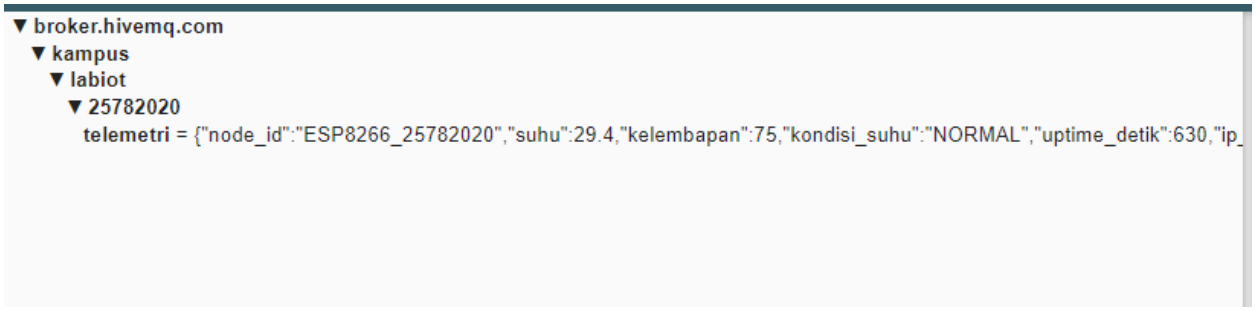
```
// ================================
// 11. SETUP
// ================================
void setup() {
  Serial.begin(115200);
  pinMode(LEDPIN, OUTPUT);
  digitalWrite(LEDPIN, LOW);
  dht.begin();
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  client.setServer(mqttServer, mqttPort);
  client.setCallback(callback);
  Serial.println("SISTEM INDUSTRI IoT DIMULAI");
}
// ================================
// 12. LOOP UTAMA
// ================================
void loop() {
  connectWiFi();
  reconnectMQTT();
  if (client.connected()) {
    client.loop();
  }
```

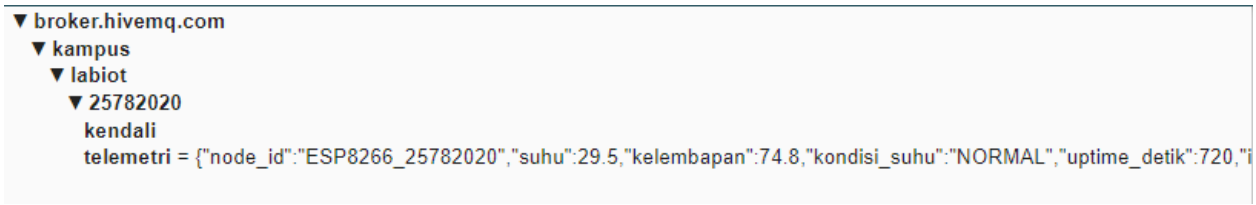
```
    readSensor();
    publishTelemetry();
    updateLED();
}
```






▼ broker.hivemq.com
▼ kampus
▼ labiot
▼ 25782020
telemetri = {"node_id":"ESP8266_25782020","suhu":29.3,"kelembapan":75,"kondisi_suhu":"NORMAL","uptime_detik":520,"ip_


▼ broker.hivemq.com
▼ kampus
▼ labiot
▼ 25782020
telemetri = {"node_id":"ESP8266_25782020","suhu":29.4,"kelembapan":75,"kondisi_suhu":"NORMAL","uptime_detik":630,"ip_


▼ broker.hivemq.com
▼ kampus
▼ labiot
▼ 25782020
kendali
telemetri = {"node_id":"ESP8266_25782020","suhu":29.5,"kelembapan":74.8,"kondisi_suhu":"NORMAL","uptime_detik":720,"i